

Skill 8

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_OUTPUT 4096

void expand_variables(const char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        if (input[i] == '$')
        {
            i++;

            /* Handle ${VARIABLE} */
            if (input[i] == '{')
            {
                i++;

                char variable[256];
                int k = 0;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 255)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == '}')
                    i++;

                char *value = getenv(variable);

                if (value != NULL)
```

```

{
    for (int x = 0;
        value[x] != '\0' &&
        j < MAX_OUTPUT - 1;
        x++)
    {
        output[j++] = value[x];
    }
}
else
{
    printf("Undefined variable: %s\n",
        variable);
}
}

/* Handle $VARIABLE */
else if (isalpha((unsigned char)input[i]) ||
    input[i] == '_')
{
    char variable[256];
    int k = 0;

    while ((isalnum((unsigned char)input[i]) ||
        input[i] == '_') &&
        k < 255)
    {
        variable[k++] = input[i++];
    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL)
    {
        for (int x = 0;
            value[x] != '\0' &&
            j < MAX_OUTPUT - 1;
            x++)
        {
            output[j++] = value[x];
        }
    }
}

```

```

        else
        {
            printf("Undefined variable: %s\n",
                variable);
        }
    }
    else
    {
        output[j++] = '$';
    }
}
else
{
    output[j++] = input[i++];
}
}

output[j] = '\0';
}

int main()
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Variable Expansion Shell\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        expand_variables(input, output);

        printf("Expanded: %s\n", output);
    }
}

```

```

}

return 0;
}

adithya@LAPTOP-87A78KDI:~/OS$ nano variable_expansion.c
adithya@LAPTOP-87A78KDI:~/OS$ gcc variable_expansion.c -o variable_expansion
adithya@LAPTOP-87A78KDI:~/OS$ ./variable_expansion
Variable Expansion Shell
Type 'exit' to quit.

myshell$ myshell$ $HOME
Expanded: myshell$ /home/adithya
myshell$ myshell$ $USER
Expanded: myshell$ adithya
myshell$ myshell$ $PATH
Expanded: myshell$ /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApp
s/MicrosoftCorporationII.WindowsSubsystemForLinux_2.7.8.0_x64_8wekyb3d8bbwe:/mnt/c/Python314/Scripts:/mnt/c/Python314:/mnt/c/Program Files/Common Files/O
racle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/WINDOW
S/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files
/neofetch-win:/mnt/c/Program Files/Git/cmd:/mnt/c/Users/Adithya/AppData/Local/Programs/Python/Python311:/mnt/c/Users/Adithya/AppData/Local/Programs/Python
/Python311/Scripts:/mnt/c/Program Files/nodejs:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/Program Files/Docker/Docker/resources/bin:/mnt/c/Users/Adithya/Dow
nloads:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/Adithya/.local/bin:/mnt/c/Users/Adithya/AppData/Local/Programs/Python/Launcher:/mnt/c/U
sers/Adithya/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/Adithya/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/Adithya/AppData/Local/Python
/bin:/mnt/c/Users/Adithya/AppData/Local/Programs/Antigravity/bin:/mnt/c/Users/Adithya/AppData/Local/Programs/Ollama:/mnt/c/Users/Adithya/AppData/Roaming/npm
:/mnt/c/Users/Adithya/AppData/Local/Programs/Antigravity IDE/bin:/mnt/c/Program Files (x86)/MongoDB Atlas CLI:/snap/bin
myshell$ myshell$ ${HOME}
Expanded: myshell$ /home/adithya
myshell$ myshell$ Hello $USER
Expanded: myshell$ Hello adithya
myshell$ |

```

PART 2

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

```

```

#define MAX_INPUT 1024
#define MAX_ARGS 64

```

```

typedef int (*BuiltinFunction)(char **args);

```

```

int builtin_pwd(char **args)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        printf("%s\n", cwd);
        return 0;
    }

    perror("pwd");
    return 1;
}

```

```

int builtin_cd(char **args)
{

```

```

if (args[1] == NULL)
{
    char *home = getenv("HOME");

    if (home == NULL)
    {
        fprintf(stderr, "HOME not set\n");
        return 1;
    }

    if (chdir(home) != 0)
    {
        perror("cd");
        return 1;
    }
}
else
{
    if (chdir(args[1]) != 0)
    {
        perror("cd");
        return 1;
    }
}

return 0;
}

```

```

int builtin_echo(char **args)
{
    for (int i = 1; args[i] != NULL; i++)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");
    }

    printf("\n");

    return 0;
}

```

```

int builtin_export(char **args)

```

```

{
    if (args[1] == NULL)
    {
        fprintf(stderr,
            "Usage: export NAME=VALUE\n");
        return 1;
    }

    char *equal = strchr(args[1], '=');

    if (equal == NULL)
    {
        fprintf(stderr,
            "Usage: export NAME=VALUE\n");
        return 1;
    }

    *equal = '\0';

    char *name = args[1];
    char *value = equal + 1;

    if (setenv(name, value, 1) != 0)
    {
        perror("export");
        return 1;
    }

    return 0;
}

```

```

int builtin_unset(char **args)
{
    if (args[1] == NULL)
    {
        fprintf(stderr,
            "Usage: unset NAME\n");
        return 1;
    }

    if (unsetenv(args[1]) != 0)
    {
        perror("unset");
        return 1;
    }
}

```

```

    }

    return 0;
}

int builtin_exit(char **args)
{
    (void)args;
    return -1;
}

typedef struct
{
    const char *name;
    BuiltinFunction function;
} Builtin;

Builtin builtins[] =
{
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"echo", builtin_echo},
    {"export", builtin_export},
    {"unset", builtin_unset},
    {"exit", builtin_exit}
};

#define BUILTIN_COUNT \
    (sizeof(builtins) / sizeof(builtins[0]))

BuiltinFunction find_builtin(const char *command)
{
    for (size_t i = 0; i < BUILTIN_COUNT; i++)
    {
        if (strcmp(command, builtins[i].name) == 0)
        {
            return builtins[i].function;
        }
    }

    return NULL;
}

int parse_input(char *input, char **args)

```

```

{
    int count = 0;

    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }

    args[count] = NULL;

    return count;
}

```

```

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Built-in Dispatch Shell\n");
    printf("Available built-ins:\n");
    printf("cd pwd echo export unset exit\n\n");

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        parse_input(input, args);

        BuiltinFunction function =
            find_builtin(args[0]);

        if (function == NULL)

```

```

{
    printf("myshell: command not found: %s\n",
        args[0]);
    continue;
}

int result = function(args);

if (result == -1)
    break;
}

return 0;
}

```

```

myshell$ exit
adithya@LAPTOP-07A78KDI:~/OS$ nano builtin_dispatch.c
adithya@LAPTOP-07A78KDI:~/OS$ gcc builtin_dispatch.c -o builtin_dispatch
adithya@LAPTOP-07A78KDI:~/OS$ ./builtin_dispatch
Built-in Dispatch Shell
Available built-ins:
cd pwd echo export unset exit

myshell$ pwd
/home/adithya/OS
myshell$ myshell$ cd ..
myshell: command not found: myshell$
myshell$ myshell$ cd OS
myshell: command not found: myshell$
myshell$ ^C
adithya@LAPTOP-07A78KDI:~/OS$ myshell$ cd OS
myshell$: command not found
adithya@LAPTOP-07A78KDI:~/OS$ myshell$ cd OS
myshell$: command not found
adithya@LAPTOP-07A78KDI:~/OS$ ./builtin_dispatch
Built-in Dispatch Shell
Available built-ins:
cd pwd echo export unset exit

myshell$ myshell$ unset NAME
myshell: command not found: myshell$
myshell$ |

```